VIT-AP UNIVERSITY

NAME: J. VEERA BHADRA

REG NO: 23BCE9780

LAB-6,SLOT :- L37+L38

1.Create a database called ‘vehicles’ and write a MongoDB query to select database as “vehicles”.

```
> use vehicle
< switched to db vehicle
> db.cars.insertMany([
  {
      vehicle_id: 1,
      brand: "Toyota",
      model: "Camry",
      year: 2022,
      color: "Black"
  },
  {
      vehicle_id: 2,
      brand: "Honda",
      model: "Civic",
      year: 2021,
      color: "White"
  },
  {
      vehicle_id: 3,
      brand: "Ford",
      model: "Mustang",
      year: 2023,
      color: "Red"
  }
  ])
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b1763c07958e428ff05939'),
    '1': ObjectId('69b1763c07958e428ff0593a'),
    '2': ObjectId('69b1763c07958e428ff0593b')
  }
}
```

2. Write a MongoDB query to display all the databases.

```
show dbs
admin      40.00 KiB
config    100.00 KiB
local      40.00 KiB
vehicle    72.00 KiB
```

3. Create a collection called ‘two_wheelers’. (use capping) and Create a collection called ‘four_wheelers’.

```
use vehicles

db.createCollection("two_wheelers", { capped: true, size: 5242880, max: 100 })

db.createCollection("four_wheelers")

show collections
```

4. Add 5 two-wheeler details to the collection named ‘two_wheelers’. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc,125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
> use vehicles

 db.two_wheelers.insertMany([
 {
 bike_name: "Hero Splendor",
 model: "gear",
 category: "100cc",
 colors_available: ["red","black","blue"],
 manufacturer: "Hero",
 performance: 8,
 timestamp: new Date("2022-05-10"),
 price: 75000
 },
 {
 bike_name: "Honda Activa",
 model: "gearless",
 category: "125cc",
 colors_available: ["black","white","blue"],
 manufacturer: "Honda",
 performance: 9,
 timestamp: new Date("2023-02-15"),
 price: 85000
 },
```

```
{
bike_name: "Bajaj Pulsar",
model: "gear",
category: "150cc",
colors_available: ["red","black","sport red"],
manufacturer: "Bajaj",
performance: 8,
timestamp: new Date("2021-08-20"),
price: 110000
},
{
bike_name: "TVS Apache",
model: "gear",
category: "200cc",
colors_available: ["black","blue","red"],
manufacturer: "TVS",
performance: 9,
timestamp: new Date("2024-01-05"),
price: 145000
},
{
bike_name: "Yamaha Ray ZR",
model: "gearless",
category: "125cc",
colors_available: ["blue","black","red"],
manufacturer: "Yamaha",
performance: 8,
timestamp: new Date("2023-06-12"),
```

5. Add 5 four-wheeler details to the collection named ‘four_wheelers’. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
db.four_wheelers.insertMany([
{
vehicle_name: "Maruti Swift",
model: "own",
category: "car",
variants: ["vxi","zxi","petrol"],
manufacturer: "Maruti Suzuki",
performance: 8,
timestamp: new Date("2022-03-15"),
price: 700000
},
{
vehicle_name: "Tata Ace",
model: "commercial",
category: "mini truck",
variants: ["diesel","petrol"],
manufacturer: "Tata",
performance: 7,
timestamp: new Date("2021-06-10"),
price: 450000
},
{
vehicle_name: "Ashok Leyland Bus",
model: "commercial",
category: "bus",
variants: ["diesel"],
manufacturer: "Ashok Leyland",
performance: 8,
```

```
  model: "commercial",
  category: "lorry",
  variants: ["diesel","zxi"],
  manufacturer: "Mahindra",
  performance: 8,
  timestamp: new Date("2023-01-12"),
  price: 900000
  },
  {
  vehicle_name: "Volvo FH16",
  model: "commercial",
  category: "heavy truck",
  variants: ["diesel"],
  manufacturer: "Volvo",
  performance: 9,
  timestamp: new Date("2024-04-05"),
  price: 8500000
  }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69b177ed07958e428ff0593e'),
      '1': ObjectId('69b177ed07958e428ff0593f'),
      '2': ObjectId('69b177ed07958e428ff05940'),
      '3': ObjectId('69b177ed07958e428ff05941'),
      '4': ObjectId('69b177ed07958e428ff05942')
    }
  }
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

```
> db.two_wheelers.find()

  db.four_wheelers.find()
< {
    _id: ObjectId('69b177ed07958e428ff0593e'),
    vehicle_name: 'Maruti Swift',
    model: 'own',
    category: 'car',
    variants: [
      'vxi',
      'zxi',
      'petrol'
    ],
    manufacturer: 'Maruti Suzuki',
    performance: 8,
    timestamp: 2022-03-15T00:00:00.000Z,
    price: 700000
  }
  {
    _id: ObjectId('69b177ed07958e428ff0593f'),
    vehicle_name: 'Tata Ace',
    model: 'commercial',
    category: 'mini truck',
    variants: [
      'diesel',
      'petrol'
    ],
```

```
  manufacturer: 'Tata',
  performance: 7,
  timestamp: 2021-06-10T00:00:00.000Z,
  price: 450000
}
{
  _id: ObjectId('69b177ed07958e428ff05940'),
  vehicle_name: 'Ashok Leyland Bus',
  model: 'commercial',
  category: 'bus',
  variants: [
    'diesel'
  ],
  manufacturer: 'Ashok Leyland',
  performance: 8,
  timestamp: 2020-09-20T00:00:00.000Z,
  price: 2500000
}
{
  _id: ObjectId('69b177ed07958e428ff05941'),
  vehicle_name: 'Mahindra Bolero Pickup',
  model: 'commercial',
  category: 'lorry',
  variants: [
    'diesel',
    'zxi'
  ],
```

```
    manufacturer: 'Mahindra',
    performance: 8,
    timestamp: 2023-01-12T00:00:00.000Z,
    price: 900000
  }
  {
    _id: ObjectId('69b177ed07958e428ff05942'),
    vehicle_name: 'Volvo FH16',
    model: 'commercial',
    category: 'heavy truck',
    variants: [
      'diesel'
    ],
    manufacturer: 'Volvo',
    performance: 9,
    timestamp: 2024-04-05T00:00:00.000Z,
    price: 8500000
  }
```

7.Write a MongoDB query to display only vehicle name and price in all the collection of the database

```
db.two_wheelers.find({}, {bike_name: 1, price: 1, _id: 0})

db.four_wheelers.find({}, {vehicle_name: 1, price: 1, _id: 0})
{
  vehicle_name: 'Maruti Swift',
  price: 700000
}
{
  vehicle_name: 'Tata Ace',
  price: 450000
}
{
  vehicle_name: 'Ashok Leyland Bus',
  price: 2500000
}
{
  vehicle_name: 'Mahindra Bolero Pickup',
  price: 900000
}
{
  vehicle_name: 'Volvo FH16',
  price: 8500000
}
```

8. Write a MongoDB query to display two_wheelers from a particular company

```
  }
> db.two_wheelers.find({manufacturer: "Honda"})
<
```

9.Write a MongoDB query to display four_wheelers available in diesel variants

```
> db.two_wheelers.find({manufacturer: "Honda"})
<
> db.four_wheelers.find({variants: "diesel"})
< {
    _id: ObjectId('69b177ed07958e428ff0593f'),
    vehicle_name: 'Tata Ace',
    model: 'commercial',
    category: 'mini truck',
    variants: [
      'diesel',
      'petrol'
    ],
    manufacturer: 'Tata',
    performance: 7,
    timestamp: 2021-06-10T00:00:00.000Z,
    price: 450000
  }
  {
    _id: ObjectId('69b177ed07958e428ff05940'),
    vehicle_name: 'Ashok Leyland Bus',
    model: 'commercial',
    category: 'bus',
    variants: [
      'diesel'
    ],
    manufacturer: 'Ashok Leyland',
    performance: 8,
    timestamp: 2020-09-20T00:00:00.000Z,
    price: 2500000
```

```
}
{
  _id: ObjectId('69b177ed07958e428ff05941'),
  vehicle_name: 'Mahindra Bolero Pickup',
  model: 'commercial',
  category: 'lorry',
  variants: [
    'diesel',
    'zxi'
  ],
  manufacturer: 'Mahindra',
  performance: 8,
  timestamp: 2023-01-12T00:00:00.000Z,
  price: 900000
}
{
  _id: ObjectId('69b177ed07958e428ff05942'),
  vehicle_name: 'Volvo FH16',
  model: 'commercial',
  category: 'heavy truck',
  variants: [
    'diesel'
  ],
  manufacturer: 'Volvo',
  performance: 9,
  timestamp: 2024-04-05T00:00:00.000Z,
  price: 8500000
}
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

```
> db.two_wheelers.find(
  { performance: { $gt: 5 } },
  { bike_name: 1, category: 1, manufacturer: 1, _id: 0 }
  )

  db.four_wheelers.find(
  { performance: { $gt: 5 } },
  { vehicle_name: 1, category: 1, manufacturer: 1, _id: 0 }
  )
< {
    vehicle_name: 'Maruti Swift',
    category: 'car',
    manufacturer: 'Maruti Suzuki'
  }
  {
    vehicle_name: 'Tata Ace',
    category: 'mini truck',
    manufacturer: 'Tata'
  }
  {
    vehicle_name: 'Ashok Leyland Bus',
    category: 'bus',
    manufacturer: 'Ashok Leyland'
  }
  {
    vehicle_name: 'Mahindra Bolero Pickup',
    category: 'lorry',
    manufacturer: 'Mahindra'
```

```
  {
    vehicle_name: 'Volvo FH16',
    category: 'heavy truck',
    manufacturer: 'Volvo'
  }
```

2. Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called ‘animal’ and write a MongoDB query to select database as ‘animal’.

```
> use animal
< switched to db animal
animal>
```

2. Write a MongoDB query to display all the databases.

```
> show dbs
< admin      40.00 KiB
  config    104.00 KiB
  local      40.00 KiB
  vehicle    72.00 KiB
  vehicles   40.00 KiB
animal>
```

3. Create a collection called ‘wild_animals’.(use capping) and Create a collection called ‘domestic_animals’.

```
db.createCollection("wild_animals", { capped: true, size: 5242880, max: 100 });

db.createCollection("domestic_animals");

db.getCollectionNames();
[ 'domestic_animals', 'wild_animals' ]
```

4. Add 5 wild_animal details to the collection named ‘wild_animals’. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and

expenses.

```
> db.wild_animals.insertMany([
  {
  animal_name: "Lion",
  nature: "harm",
  favorite_foods: ["meat","deer"],
  care_taker_name: "Ravi",
  life_span: 15,
  timestamp: new Date("2021-05-10"),
  expenses: 50000
  },
  {
  animal_name: "Tiger",
  nature: "harm",
  favorite_foods: ["meat","rabbits","deer"],
  care_taker_name: "Suresh",
  life_span: 16,
  timestamp: new Date("2022-03-18"),
  expenses: 60000
  },
  {
  animal_name: "Elephant",
  nature: "harmless",
  favorite_foods: ["grass","fruits","sugarcane"],
  care_taker_name: "Mahesh",
  life_span: 60,
  timestamp: new Date("2020-11-25"),
  expenses: 70000
  },
```

```
{
animal_name: "Leopard",
nature: "harm",
favorite_foods: ["meat","deer"],
care_taker_name: "Ramesh",
life_span: 14,
timestamp: new Date("2023-01-12"),
expenses: 45000
},
{
animal_name: "Bear",
nature: "harm",
favorite_foods: ["fish","honey","fruits"],
care_taker_name: "Ganesh",
life_span: 20,
timestamp: new Date("2021-08-30"),
expenses: 40000
}
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b180f207958e428ff05943'),
    '1': ObjectId('69b180f207958e428ff05944'),
    '2': ObjectId('69b180f207958e428ff05945'),
    '3': ObjectId('69b180f207958e428ff05946'),
    '4': ObjectId('69b180f207958e428ff05947')
  }
}
```

5. Add 5 domestic-animal details to the collection named ‘domestic_animals’. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
> db.domestic_animals.insertMany([
  {
  animal_name: "Dog",
  gender: "male",
  favorite_foods: ["meat","bones"],
  animal_petname: "Tommy",
  life_span: 13,
  timestamp: new Date("2022-04-12"),
  expenses: 5000
  },
  {
  animal_name: "Cat",
  gender: "female",
  favorite_foods: ["fish","milk"],
  animal_petname: "Kitty",
  life_span: 15,
  timestamp: new Date("2021-07-18"),
  expenses: 3000
  },
  {
  animal_name: "Cow",
  gender: "female",
  favorite_foods: ["grass","hay"],
  animal_petname: "Lakshmi",
  life_span: 20,
  timestamp: new Date("2020-09-25"),
  expenses: 7000
  },
  {
```

```
{
animal_name: "Goat",
gender: "male",
favorite_foods: ["grass","leaves"],
animal_petname: "Bunny",
life_span: 12,
timestamp: new Date("2023-01-10"),
expenses: 2500
},
{
animal_name: "Rabbit",
gender: "female",
favorite_foods: ["carrot","grass"],
animal_petname: "Snowy",
life_span: 9,
timestamp: new Date("2022-11-05"),
expenses: 2000
}
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b1817807958e428ff05948'),
    '1': ObjectId('69b1817807958e428ff05949'),
    '2': ObjectId('69b1817807958e428ff0594a'),
    '3': ObjectId('69b1817807958e428ff0594b'),
    '4': ObjectId('69b1817807958e428ff0594c')
  }
}
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

```
> db.wild_animals.find()

  db.domestic_animals.find()
< {
    _id: ObjectId('69b1817807958e428ff05948'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [
      'meat',
      'bones'
    ],
    animal_petname: 'Tommy',
    life_span: 13,
    timestamp: 2022-04-12T00:00:00.000Z,
    expenses: 5000
  }
  {
    _id: ObjectId('69b1817807958e428ff05949'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [
      'fish',
      'milk'
    ],
    animal_petname: 'Kitty',
    life_span: 15,
    timestamp: 2021-07-18T00:00:00.000Z,
    expenses: 3000
  }
```

```
{
  _id: ObjectId('69b1817807958e428ff0594a'),
  animal_name: 'Cow',
  gender: 'female',
  favorite_foods: [
    'grass',
    'hay'
  ],
  animal_petname: 'Lakshmi',
  life_span: 20,
  timestamp: 2020-09-25T00:00:00.000Z,
  expenses: 7000
}
{
  _id: ObjectId('69b1817807958e428ff0594b'),
  animal_name: 'Goat',
  gender: 'male',
  favorite_foods: [
    'grass',
    'leaves'
  ],
  animal_petname: 'Bunny',
  life_span: 12,
  timestamp: 2023-01-10T00:00:00.000Z,
  expenses: 2500
}
{
  _id: ObjectId('69b1817807958e428ff0594c'),
  animal_name: 'Rabbit',
```

```
{
  _id: ObjectId('69b1817807958e428ff0594c'),
  animal_name: 'Rabbit',
  gender: 'female',
  favorite_foods: [
    'carrot',
    'grass'
  ],
  animal_petname: 'Snowy',
  life_span: 9,
  timestamp: 2022-11-05T00:00:00.000Z,
  expenses: 2000
}
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

```
> db.wild_animals.find({}, {animal_name: 1, expenses: 1, _id: 0})

  db.domestic_animals.find({}, {animal_name: 1, expenses: 1, _id: 0})
< {
    animal_name: 'Dog',
    expenses: 5000
  }
  {
    animal_name: 'Cat',
    expenses: 3000
  }
  {
    animal_name: 'Cow',
    expenses: 7000
  }
  {
    animal_name: 'Goat',
    expenses: 2500
  }
  {
    animal_name: 'Rabbit',
    expenses: 2000
  }
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

```
> db.domestic_animals.find({ life_span: 12 })
< {
    _id: ObjectId('69b1817807958e428ff0594b'),
    animal_name: 'Goat',
    gender: 'male',
    favorite_foods: [
      'grass',
      'leaves'
    ],
    animal_petname: 'Bunny',
    life_span: 12,
    timestamp: 2023-01-10T00:00:00.000Z,
    expenses: 2500
  }
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

```
> db.wild_animals.find({ care_taker_name: "Ravi" })
< {
    _id: ObjectId('69b180f207958e428ff05943'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [
      'meat',
      'deer'
    ],
    care_taker_name: 'Ravi',
    life_span: 15,
    timestamp: 2021-05-10T00:00:00.000Z,
    expenses: 50000
  }
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

```
> db.wild_animals.find(
  { life_span: { $gt: 5 } },
  { animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0 }
  )

  db.domestic_animals.find(
  { life_span: { $gt: 5 } },
  { animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0 }
  )
< {
    animal_name: 'Dog',
    favorite_foods: [
      'meat',
      'bones'
    ],
    expenses: 5000
  }
  {
    animal_name: 'Cat',
    favorite_foods: [
      'fish',
      'milk'
    ],
    expenses: 3000
  }
  {
```

```
{
  animal_name: 'Cow',
  favorite_foods: [
    'grass',
    'hay'
  ],
  expenses: 7000
}
{
  animal_name: 'Goat',
  favorite_foods: [
    'grass',
    'leaves'
  ],
  expenses: 2500
}
{
  animal_name: 'Rabbit',
  favorite_foods: [
    'carrot',
    'grass'
  ],
  expenses: 2000
}
```